Web-Chat Agent Bridge — documento di progetto

Oct 14, 2026 · @Davide

Sintesi

L'obiettivo è un tool open source che si avvia come un agente da terminale ma usa come motore una chat web in abbonamento, a partire da Gemini con Gemini 4 Argon. Il browser lavora in background: l'utente parla con l'agente e non vede mai la pagina.

L'idea di base, chat web più esecutore locale, esiste già in vari progetti. Il valore aggiunto sta in tre cose che oggi mancano insieme:

- un'esperienza indistinguibile da un agente nativo, con chat nascosta;
- allegati multimodali veri, caricati come file e non incollati come testo;
- un livello di affidabilità sopra il protocollo dei tool.

Nome provvisorio: Web-Chat Agent Bridge.

Problema e motivazione

Il modello migliore è spesso raggiungibile in chat ma non come agente. Dal 13 ottobre 2026 Gemini 4 Argon è nell'app Gemini e in AI Mode per gli abbonati AI Pro e Ultra. In Antigravity, l'ambiente di coding agentico, è disponibile solo per Ultra.

- **Chat:** l'abbonamento è già pagato, ma lì il modello non può toccare file né eseguire comandi.
- **API:** 2 $ per milione di token in input e 10 $ in output fino al 30 novembre 2026, poi 4 $ e 20 $.
- **Obiettivo del tool:** riusare la chat come motore di un agente locale.

Per un singolo utente l'API può costare meno del tempo speso a mantenere un bridge. Il tool ha quindi senso in tre casi: se copre più piattaforme, se serve a chi non ha accesso all'API, oppure come progetto open source e da portfolio.

Stato dell'arte

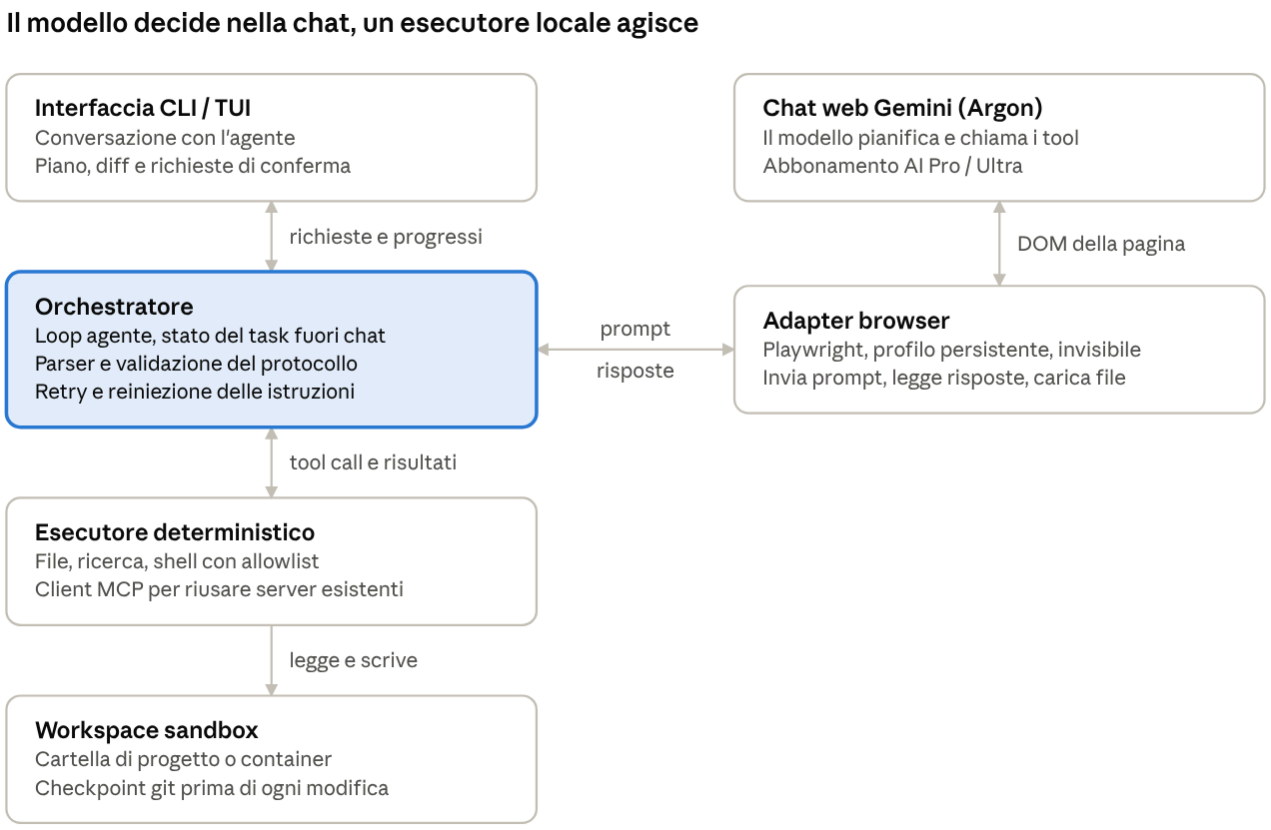
Esistono già diversi pezzi, ma nessuno dei tool trovati offre insieme chat nascosta, multimodalità e affidabilità. La ricerca è stata rapida e va verificata.

Tool	Cosa fa	Limite rispetto all'idea
MCP SuperAssistant	Estensione che rileva le chiamate MCP nella chat e le esegue tramite un proxy locale; supporta Gemini, con modalità di esecuzione e invio automatici	Si lavora dentro la pagina; il formato delle chiamate a volte fallisce; nessun allegato multimodale documentato
chatgpt-use	Demone locale che pilota ChatGPT web in background con un protocollo di tool	Solo ChatGPT; pensato come backend per altri agenti
OpenBrowser	CLI più estensione: inietta i prompt, cattura le risposte, applica le operazioni sui file	Serve una tab della chat aperta e visibile
Gemini Coder	Estensione VS Code che inizializza la chat web e applica le risposte ai file	Non agentico per scelta
aider	Modalità copia-incolla con la web chat, anche senza API	Passaggi manuali; evita apposta l'automazione per i termini di servizio
chatgpt-local-coder, DevSpace	Server MCP locali per la Developer Mode di ChatGPT	Via ufficiale, ma solo per ChatGPT

Alternativa da considerare: invece di partire da zero, contribuire a MCP SuperAssistant, per esempio sull'adapter Gemini e sul supporto multimodale.

Architettura proposta

L'utente parla solo con la CLI. L'orchestratore traduce ogni passo in un prompt per la chat web, tramite un browser invisibile, ed esegue in locale le azioni che il modello chiede.



architettura · 6 componenti, chat web nascosta

Il loop gira tra orchestratore, adapter e chat finché il task non è chiuso. Solo l'esecutore tocca il workspace, e sempre dentro la sandbox.

Protocollo dei tool ed esecutore

Il modello decide e l'esecutore esegue in modo deterministico. Nessun LLM fa da burattino: un modello piccolo serve al massimo come fallback, per riparare una chiamata malformata.

Il modello scrive le chiamate in un blocco delimitato, una o più per risposta, ciascuna con un id:

```
{"tool": "edit", "id": 3, "path": "src/app.py", "search": "timeout=10",
"replace": "timeout=30"}
```

Tool	Cosa fa
list_dir	Elenca una cartella del workspace
read_file	Legge un file, con offset e limite di righe
write_file	Crea o sovrascrive un file

Tool	Cosa fa
edit	Modifica con search/replace, senza riscrivere il file
move / copy	Sposta o copia file
search	Cerca testo nel workspace
run	Esegue un comando di shell da una allowlist
attach	Carica un file locale nella chat come allegato
screenshot	Cattura una pagina o un'app e la invia come immagine

- **Risultati:** tornano in forma compatta con l'id della chiamata. Se troncati, lo dicono; un output lungo va come file allegato.
- **MCP:** l'esecutore funziona anche da client MCP, così si riusano server esistenti come Desktop Commander.

Livello di affidabilità

È la parte che distingue un giocattolo da uno strumento usabile, ed è la più debole nei tool esistenti.

- **Parser tollerante e validazione dello schema.** Se una chiamata è malformata, al modello parte in automatico un messaggio di correzione, con un numero massimo di tentativi.
- **Reiniezione delle istruzioni.** Il protocollo viene riproposto ogni tot turni, o quando il modello smette di usarlo dove servirebbe.
- **Stato del task fuori dalla chat.** Piano e lista dei passi vivono in locale e vengono reinseriti in forma compatta. Così la sessione sopravvive alla perdita di contesto o a una chat nuova.
- **Fine della generazione.** Va rilevata in modo robusto, cioè stream terminato e pulsante di stop sparito, con timeout.
- **Limiti d'uso.** Il tool riconosce il messaggio di limite, mette in pausa e avvisa l'utente.
- **Log di sessione completo**, per debug e replay.

Supporto multimodale

I file passano dal campo di upload della pagina come allegati veri, non come testo incollato. È il punto che i tool esistenti non documentano.

- **Verso il modello:** immagini, PDF e gli altri formati che la chat accetta, presi dal workspace.
- **Ciclo visivo:** gli screenshot di pagine o app prodotte dall'agente tornano al modello come immagini. Serve soprattutto per il front-end.
- **Dal modello:** i file generati in chat, come immagini o documenti, vengono scaricati e salvati nel workspace.
- **Limiti:** valgono dimensioni, numero e formati di upload ammessi dal piano.

Sicurezza e sandbox

Un modello che scrive file ed esegue comandi sul PC richiede regole fissate dall'inizio, non aggiunte dopo.

- **Workspace confinato:** una cartella di progetto o un container. I path esterni vengono rifiutati.
- **Permessi a livelli:** la lettura è libera. La scrittura mostra il diff. Shell e operazioni distruttive chiedono conferma, con una modalità automatica attivabile.
- **Checkpoint git** automatico prima di ogni modifica, così tutto è reversibile.
- **Prompt injection:** il contenuto letto da pagine o file è trattato come dato. Un comando che nasce da quel contenuto non parte senza conferma.
- **Credenziali:** il tool usa il profilo browser locale dell'utente, non salva token e non invia dati a terzi.

Rischi e limiti

Il rischio più serio sono i termini di servizio: secondo la documentazione di aider, la maggior parte delle chat web vieta di automatizzare l'incolla nella chat e la copia della risposta.

Rischio	Impatto	Mitigazione
Termini di servizio, con possibile ban dell'account	Alto	Avviso chiaro nel README; modalità copia-incolla assistita e conforme; backend API come alternativa
Fragilità del DOM: la pagina cambia e il bridge si rompe	Medio-alto	Un adapter isolato per piattaforma, selettori centralizzati, test automatici di base
Latenza di ogni passo	Medio	Feedback continuo nell'interfaccia; più chiamate per risposta
Limiti d'uso del piano consumer	Medio	Rilevamento e pausa; passaggio all'API

Rischio	Impatto	Mitigazione
Contesto gestito dal provider, non dal tool	Medio	Stato del task esterno e reiniezione delle istruzioni
Concorrenza dei tool esistenti	Medio	Puntare sui tre punti di differenza, o contribuire a MCP SuperAssistant

Roadmap

Prima di tutto conviene verificare con uno spike che Gemini web si possa pilotare in modo stabile; il resto dipende da quel risultato.

1. **Spike.** Pilotare Gemini web con Playwright e un profilo persistente: invio del prompt, attesa della fine, lettura della risposta, upload di un file.
   - Criterio di uscita: 20 andata e ritorno consecutivi senza errori.
2. **MVP.** CLI, protocollo, esecutore per i file (list, read, write, edit), loop automatico e checkpoint git.
   - Criterio di uscita: completare un task reale su un repo piccolo, per esempio aggiungere un test.
3. **Affidabilità.** Validazione, retry, reiniezione delle istruzioni, stato esterno, gestione dei limiti d'uso.
   - Criterio di uscita: misurare il tasso di chiamate malformate e di task completati su 10–20 task.
4. **Multimodale.** Upload dal workspace e ciclo con gli screenshot.
5. **Sicurezza e shell.** Sandbox, permessi a livelli, shell con allowlist.
6. **Altre piattaforme.** Adapter per ChatGPT e AI Studio, più un backend API alternativo.
7. **Rilascio open source.** README con l'avviso sui termini di servizio, demo, guida per scrivere nuovi adapter.

Domande aperte e prossimi passi

- [ ] Linguaggio: TypeScript, più vicino all'ecosistema MCP e a Playwright, oppure Python?
- [ ] Forma: CLI/TUI o app desktop?
- [ ] Solo Gemini all'inizio o più piattaforme da subito?
- [ ] Accettare il rischio sui termini di servizio, o partire dalla modalità copia-incolla conforme?
- [ ] Costruire da zero, oppure fare un fork o contribuire a MCP SuperAssistant?

- [ ] Definire un set di task per misurare l'affidabilità, utile anche come eventuale angolo di ricerca.

I prossimi passi sono un'ora di ricerca su GitHub per confermare il vuoto, con query come "web chat agent backend" e "gemini web agent", e poi lo spike della roadmap.